

AgilA32

Developed by Alexander Co Abad with the help of Claude AI (Sonnet 5.5)

An RV32I CPU written for this project, with RP2040-compatible PIO state machines on one die: a firmware-programmable protocol emulator for IHP SG13 CMOS5L / Tiny Tapeout (8x4 tiles)

Document	Project documentation for review
Author	Alexander Co Abad, Department of Electronics, Computer and Electrical Engineering, De La Salle University, Manila
Repository	github.com/alexandercoabad/AgilA32_PIO (snapshot of 7 Oct 2026, features 1-21)
Revision	10 Oct 2026: organisers' update incorporated (judging criteria, submission list, reset and power rules, GitHub time limit) 30 Oct 2026: formal proofs and random tests added (Appendix B); Sections 1, 2, 8, 9 updated.
Purpose	Entry material for the Jane Street protocol-emulator ASIC competition (submission deadline 18 Jan 2027)
Target	IHP SG13 CMOS5L via Tiny Tapeout, 8x4 tiles, 24 MHz, open-source flow (Yosys / LibreLane)
Originality	The CPU is written for this project, not derived from an existing open-source core; it follows the public RV32I specification, and the PIO follows the public RP2040 datasheet.
Status	Simulation, post-layout simulation and RTL formal proofs only. Nothing has been measured on silicon.

How to read this document. Every number is traceable to a file in the repository, a CI run, or a log named in the text. Where something is simulated, modelled, estimated or not done, the text says so in that place, and Section 9 gathers the limitations in one list. Section 8 maps each claim to the evidence behind it.

Contents

1. Executive summary 4

Headline figures . . . 4

2. Goal and fit to the competition brief 5

Lineage . . . 5

Submission requirements and where each stands . . . 6

3. System architecture 6

3.1 CPU . . . 6

3.2 Memory map (8-bit address space) . . . 7

3.3 Boot and reprogramming . . . 7

3.4 Pinout . . . 7

3.5 Reset, power-on state and reload . . . 8

4. The PIO block 8

4.1 Host interface and registers . . . 8

4.2 Design decisions that shape the block . . . 9

4.3 Host tooling . . . 9

5. Protocol coverage 10

6. Physical implementation and portability 11

6.1 Hardening runs . . . 11

6.2 Timing at 24 MHz . . . 12

6.3 Portability: PDK-agnostic RTL, no macros . . . 12

7. Verification 13

7.1 Method and what it can and cannot show . . . 13

7.2 Test inventory . . . 14

7.3 Mutation sweeps . . . 15

7.4 Post-layout (gate-level) tests . . . 15

7.5 Continuous integration . . . 17

8. Claims and the evidence behind them 18

9. Limitations and open work 19

9.1 Architecture . . . 19

9.2 Protocol-specific . . . 19

9.3 Verification and evidence . . . 19

9.4 Open work, in suggested order . . . 20

10. Reproducing the results 20

Repository layout . . . 21

11. Use of AI tools 21

Appendix A. Feature history 22

Sources used for this document 22

Appendix B. Formal proofs and random tests 23

B.1 Formal proofs of the PIO FIFO and state machine 23

B.2 Result on GitHub 23

B.3 Do the proofs bite? 24

B.4 Limits of the formal work 24

B.5 Constrained-random protocol tests 24

B.6 Reproducing 24

1. Executive summary

AgilA32 is a small system-on-chip built to answer one question: *can a chip learn new protocols after fabrication, with no host microcontroller?* It puts an RV32I CPU written for this project next to two state machines that execute the RP2040 PIO instruction set. The CPU loads PIO programs from flash or from a GPIO bootloader, can replace them at run time, and can halt itself (EBREAK) while the PIO keeps the protocol running alone. On the RP2040 the programmer is an external Cortex-M0+; here it is on the die.

The design is plain synthesizable Verilog (about 2,900 lines in `src/`), uses no PDK cells and no macros, and hardens at 8x4 tiles (the planned maximum in the brief) on IHP SG13 CMOS5L; it did not route at 6x4.

Headline figures

Item	Value	Where it comes from
Protocols demonstrated in simulation	UART TX/RX, SPI master modes 0-3, I2C master / slave / multi-master, JTAG, SWD, CAN 2.0A/B (CPU-assisted), low-speed USB host, PS/2 receiver, WS2812 (two programs), 1-Wire, VGA 640x480	Section 5, `README.md`
Contest baseline (UART, SPI, I2C)	All three done, each with a CPU-driven end-to-end demo	Section 5
Stretch goals	Low-speed USB: done in simulation (host side, narrow receive tolerance). 10BASE-T: not attempted	Section 5, 9
PIO resources	2 state machines, 32-word shared instruction memory, 4-deep FIFOs, pins 0-9	`src/pio.v`
PIO programs in the repository	23 (19 protocol programs + 2 smoke-test + 2 signature-test programs)	`pio/`
Cocotb protocol tests	120 passing (77 + 18 SWD + 25 CAN) in GitHub run #26 (10 min 5 s; cocotb 444.4 s); run #23 passed 95/95 before the CAN tests were added	pio-tests runs #26 (79c44fc) and #23 (22f824b)
Standalone Icarus benches	32 sources, 34 runs (two WS2812 firmware variants re-run the same bench), all PASS	CI logs, run #26, 7 Oct 2026
Mutation testing	221 one-line mutants over eight sweeps: 219 caught; 2 survivors in the signature sweep (Section 7.3)	`tools/` (8 sweep scripts)
Gate-level tests	13/13 passing on the placed IHP SG13 CMOS5L netlist at 8x4, with the PIO smoke and signature tests (gds run #90, push 16b8345, branch try-density58, 12-13 Oct 2026): `gl_test` 8 min 1 s, gds 4 h 34 min, precheck 4 h 47 min, whole workflow 9 h 22 min (Section 7.5)	Tiny Tapeout `gl_test` job, `results.xml`
Timing at 24 MHz	Worst setup slack +20.17 ns (slow corner); register-to-register limit about 46.5 MHz; hold +0.116 ns (fast corner)	GDS run, `docs/info.md`
Physical checks	0 Magic DRC, 0 routing DRC, 0 LVS errors, 0 antenna-violating nets	GDS run
Formal proofs	4 of 4 proofs pass: PIO FIFO and state machine	formal/, GitHub run #3
Silicon measurements	None	-

What a reviewer should weigh. Strength: breadth of protocols on one small block, and a verification approach built around peer models, CPU-driven end-to-end benches and mutation testing. Weakness: RV32I plus an RP2040-style PIO is close to the RP2040's own architecture, every result is simulation or RTL proof, and only two small PIO tests have run on the post-layout netlist: a smoke test and a wider signature test (I2C pads, most instruction forms, autopush / autopull, side-set), which passed there on 5-6 Oct 2026. The protocol programs themselves are simulated on RTL only, and the gate-level job now takes about 3 hours (Section 7.4).

2. Goal and fit to the competition brief

The Jane Street brief asks for an open-source, reprogrammable protocol-emulator chip, inspired by the RP2040 PIO and the TI PRU, on IHP CMOS5L through Tiny Tapeout. UART, SPI and I2C are the baseline; low-speed USB and 10 Mbit Ethernet are stretch goals. The organisers' update of October 2026 states that there is no fixed weighting: flexibility is the main criterion (can the chip be reprogrammed for protocols it was not designed for), followed by the protocols demonstrated, anything unique in the architecture, and the design and verification approach. A design with fewer protocols but a strong architecture and verification can be competitive. Area is limited to 6x4 tiles; the planned maximum is 8x4, and the 8x4 template is expected around mid-October. IHP SRAM macros are allowed. The brief recommends SRAM over flip-flops for instruction storage and FPGA prototyping before submission.

Brief item	How AgilA32 responds	Status
UART, SPI, I2C	Pico SDK programs run unmodified; SPI modes 0-3 and I2C master/slave/multi-master added	Done (sim)
Low-speed USB	`usb_ls.pio`, 29 of 32 words, 1.5 Mb/s exactly at 24 MHz / CLKDIV 2; token, DATA and reply capture	Done (sim), host side only
10 Mbit Ethernet	Needs a frame buffer (4-deep FIFO holds 128 of about 576 bits), at least 40 MHz and external magnetics	Not attempted
Reprogrammable after fabrication	CPU rewrites instruction memory and registers at run time; one image runs UART, then SPI, then I2C on the same state machine	Done (sim)
Open source, IHP CMOS5L, 8x4 tiles	Verilog template, LibreLane flow, hardened at 8x4 (6x4 did not route)	Done at 8x4
Verification novelty	Peer models, mutation sweeps, CPU-driven benches, post-layout smoke and signature tests, formal proofs of the PIO FIFO and state machine, constrained-random tests (Appendix B)	Done; formal and random tests, partial (App. B)
SRAM for instruction storage	Not used: the PIO instruction memory is 32 x 16-bit flip-flops, so the design carries no macros	Deliberate trade-off (area vs portability)
FPGA prototype	An FPGA workflow exists in the repository; no FPGA demo against a real peer device has been run	Open

Lineage

AgilA32 is the 32-bit successor of AgilA8, the author's earlier 8-bit Tiny Tapeout project (`github.com/alexandercoabad/AgilA8_microcontroller` on Sky130, `AgilA8_IHP` for the IHP port). The QSPI engine, GPIO bootloader, bank-switched flash execution, Timer/PWM and generic SPI peripheral are ported from AgilA8. The RV32I core and the whole PIO block are new. The line started from Pineapple ONE, a RISC-V computer built from discrete 7400-series logic.

Submission requirements and where each stands

Requirement (organisers' update, October 2026)	Status in this project
Link to a public repository with RTL, tests and a passing GDS / precheck from the Tiny Tapeout flow	RTL and tests are in the repository. gds run #65 passed, with precheck 10/10 and gate-level 13/13. The repository must be public at submission; its visibility is the author's decision
Documentation of the architecture and instruction set	This document, `docs/info.md`, and the PIO instruction set in Section 4
Assembler or toolchain, with example programs for the protocols supported	`tools/asm_pineapple.py` (RV32I), `tools/pioasm.py` (PIO) and the PIO programs of Section 5. The RV32I C flow for the 44-byte window and its examples are in `c/`; `SUBMISSION.md` lists the evidence for each requirement
Write-up of the verification approach	Section 7
Description of how AI tools were used	Section 11
Schedule	Submissions close 18 Jan 2027; selected designs are announced in early February 2027, ahead of the March 2027 CMOS5L shuttle; about four designs are planned

3. System architecture

The chip has three parts on one 8-bit memory-mapped bus: the CPU, the memory and peripheral decoder (`mem.v`) with its shared QSPI engine, and the PIO block. The address bus is only 8 bits wide because the design grew from AgilA8; the PIO therefore sits behind a two-byte window.

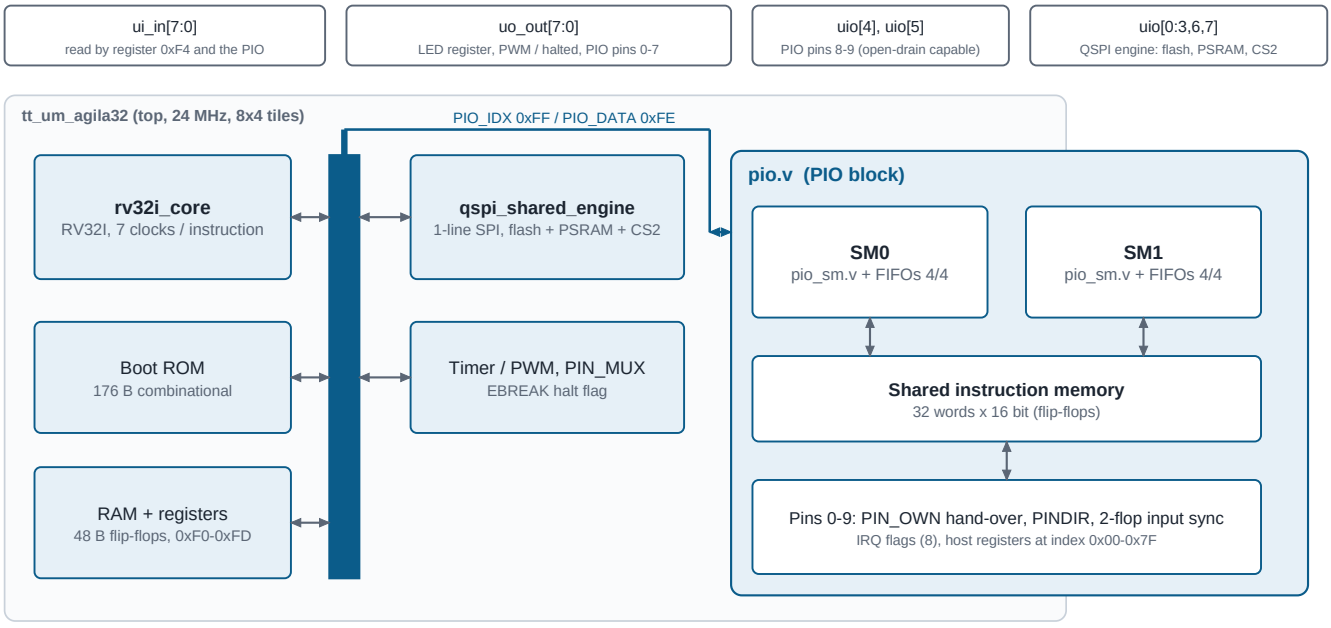


Figure 1. Block diagram. Pads are shared: pins are taken from the LED / QSPI logic only when the matching PIN_OWN bit is set.

3.1 CPU

- **Instruction set:** the full RV32I base integer set. FENCE, ECALL and EBREAK decode; there is no trap or exception handling. EBREAK is repurposed to park the core in a halted state until reset, and the halted flag can be routed to `uo_out[7]`.
- **Micro-architecture:** multi-cycle FSM (fetch, fetch_wait, decode, execute, memory, mem_wait, writeback), 7 clocks per instruction. The two wait states let the core stall on a `ready` handshake during slow external QSPI accesses.

- **Cost of a flash fetch:** external flash is read through a 44-byte execute window with bank switching; one page switch costs about 3000 clocks, so the CPU feeds about one FIFO word per 3000 clocks. This sets the limit on CPU-fed protocol speed (Section 9).
- **Toolchain:** `tools/asm_pineapple.py` is a small RV32I assembler plus `PagedAsm` for paged flash images. `tb_alu_test.v` runs every opcode it wraps through the real core.

3.2 Memory map (8-bit address space)

Address	Item	Notes
0x00-0xAF	Boot ROM, 176 B	Combinational logic (a case statement), not flip-flops, so it is present at every power-on
0xB0-0xB3	Scratch RAM, 4 B	Always on-chip
0xB4-0xDF	Loadable / executable window, 44 B	Bootloader writes here; reads come from external flash (CS0) once FLASH_MODE is set
0xE0-0xEF	External PSRAM window, 16 B	RAM A (CS1); RAM B (CS2) when PSRAM_BANK = 1; core stalls until the SPI transaction completes
0xF0 / 0xF4	LED output / switch input	Wired to `uo_out` / `ui_in`; `ui_in[0:2]` are also the bootloader DATA / CLOCK / START
0xF1	PSRAM_BANK	Selects RAM A or RAM B for the 0xE0 window
0xF2, 0xF3, 0xF5, 0xF6	TIMER_LO, TIMER_HI, TIMER_CTRL, TIMER_FLAG	Free-running 16-bit timer (ported from AgilA8)
0xF7, 0xF9, 0xFA	PWM_DUTY, PWM_CTRL, PIN_MUX	8-bit PWM; PIN_MUX picks LED / PWM / halted on `uo_out[7]`
0xF8, 0xFC	FLASH_MODE, FLASH_PAGE	Redirect the window to flash; select the 44-byte flash page
0xFB	QSPI_CTRL	SCK = clk/2, /8, /32 or /128; resets to the slowest
0xFD	SPI_DATA	Raw 8-bit SPI on CS2; needs one board modification (cut a trace on the QSPI Pmod)
0xFE / 0xFF	PIO_DATA / PIO_IDX	PIO window: IDX bits [6:0] = register, bit 7 = auto-increment; DATA = 32-bit access

3.3 Boot and reprogramming

- **Self-test:** the boot ROM writes 0xA5 to the PSRAM window, reads it back, and latches the result into `uo_out[7]`.
- **Listen loop:** a demo counter runs on `uo_out[3:0]` while START on `ui_in[2]` is polled. A bootload request streams a program into RAM and runs it immediately, with no reflash.
- **Timeout fallback:** if START never arrives, the boot ROM sets FLASH_MODE itself and runs whatever is in external flash.
- **Flash hand-off:** a 4-byte stub bootloaded over the GPIO pins hands execution to flash; programs then page through flash with FLASH_PAGE.

3.4 Pinout

Pad	Function	Pad	Function
ui[7:0]	Switches; PIO input pins 0-7 (bootloader on ui[0:2])	uio[0..3]	QSPI CS0, MOSI, MISO, SCK
uo[6:0]	LEDs; PIO pins 0-6	uio[4], uio[5]	PIO pins 8, 9 (true bidirectional, held high until owned)
uo[7]	LED / PWM / halted (PIN_MUX); PIO pin 7	uio[6], uio[7]	QSPI CS1 (PSRAM A), CS2 (PSRAM B / generic SPI)

3.5 Reset, power-on state and reload

- **What reset covers.** `rst_n` (asynchronous, active low) puts the CPU state machine, PC, instruction register and bus signals, the memory-block control registers (LED output, FLASH_MODE, FLASH_PAGE, PSRAM_BANK, QSPI divider, timer, PWM, pin mux) and the PIO host interface (index, auto-increment) into a known state.
- **What reset does not cover.** Storage arrays: the register file x1-x31, the 48-byte RAM, the 32-word PIO instruction memory and the FIFO storage. They are plain flip-flops without a reset, so on silicon they start undefined. In RTL simulation they are zeroed by `initial` blocks inside `` `ifndef SYNTHESIS ``, so simulation starts from zero where the chip does not.
- **Consequence.** A program must write a register, a RAM byte or a PIO word before it reads it. The programs written for the browser simulator follow this rule. The 13 cocotb tests and the 22 Icarus benches that instantiate the whole chip also pass with random start-up contents in the register file, RAM, PIO instruction memory and PIO FIFO memory (Section 7.5); that covers the boot ROM, the bootloader and the CPU-driven PIO programs of those benches.
- **Reload after deselection.** Per the organisers, a design that Tiny Tapeout deselects is powered down and loses its state. The program RAM, PIO instruction memory, FIFOs and registers must then be reloaded by the host (the demo board's microcontroller) through the GPIO bootloader and the CPU's PIO writes. Code kept in the external flash survives, so a flash hand-off program returns after the 4-byte stub is sent again.
- **External parts.** The pads are 3.3 V digital on the demo board. The QSPI Pmod is needed for flash and PSRAM, and pull-ups (I2C, SWD) and transceivers belong on a Pmod. No characterised pad numbers for CMOS5L were available at the organisers' update, so no pad timing is claimed.

4. The PIO block

The block executes the RP2040 PIO instruction set: JMP, WAIT, IN, OUT, PUSH/PULL, MOV, IRQ and SET, with side-set, per-instruction delay, autopush and autopull, IRQ flags and a 16.8 fractional clock divider. Each state machine runs one instruction per divided-clock tick with deterministic timing, so a protocol is a short program loaded after fabrication.

Parameter	AgilA32	RP2040 (for comparison)
State machines	2 (parameter N_SM, 1 to 4)	8 (2 blocks of 4)
Instruction memory	32 words x 16 bit, shared by all state machines	32 words per block
FIFOs	4-deep TX and RX per state machine	4-deep, joinable to 8
Pins	0-9: `uo_out[7:0]`, `uio[4]`, `uio[5]`	30 GPIO
Input synchroniser	2 flip-flops, per-pin bypass (SYNC_BYP)	Same
Programmer	On-die RV32I over a two-byte window	External Cortex-M0+
Pad direction	PINDIR is the real output enable; **PINDIR = 1 pulls the pad low** (opposite to RP2040)	PINDIR = 1 drives the pad

4.1 Host interface and registers

The register map is in the header of `src/pio.v`. The index goes into PIO_IDX; with bit 7 set the index auto-increments after each PIO_DATA access, so a whole program streams in with back-to-back stores.

Index	Register	Content
0x00	CTRL	[3:0] SM_ENABLE, [7:4] SM_RESTART (write 1, self-clearing)
0x01 / 0x02	IRQ / IRQ_FORCE	8 flags; write 1 to clear / to set
0x03	FSTAT	RXFULL, RXEMPTY, TXFULL, TXEMPTY per state machine

Index	Register	Content
0x04	PIN_OWN	[9:0] 1 = pin driven by PIO instead of the CPU / LED logic
0x05 / 0x06 / 0x07	SYNC_BYP / PINS_IN / PINS_OUT	Synchroniser bypass; input pins as seen by the SMs; output values and enables
0x08	INFO	N_SM, IMEM_DEPTH, FIFO_DEPTH, version
0x20-0x3F	IMEM	Instruction words 0-31
0x40 + 0x10 n	Per state machine n	CLKDIV, EXECCTRL, SHIFTCTRL, PINCTRL, INSTR, ADDR, TXF, RXF, FLEVEL

4.2 Design decisions that shape the block

- **PIN_OWN hand-over.** A pin leaves the LED / QSPI logic only when its PIN_OWN bit is set, so reset behaviour is identical to the pre-PIO chip and every earlier test passed unchanged. Hand-over order matters: SCK is preset in `GPIO_OUT` and a forced instruction must carry the idle level in side-set bit 12, or SCK glitches.
- **No pad-inversion override.** The Pico SDK gets SPI CPOL = 1 by inverting the SCK pad; this block has no such override, so modes 2 and 3 are separate programs with inverted side-set values.
- **Open-drain buses on a real bidirectional pad.** Pins 8-9 use PINDIR as the output enable. Because the polarity differs from the RP2040, `pio/i2c.pio` has swapped side-set values and `tools/pio_i2c.py` complements the pindir bits it sends.
- **Designed for the synthesis flow.** The first CI synthesis stalled in Yosys' SAT-based `share` pass on variable-amount shifts in `pio_sm.v`. These were replaced by fixed-stage mux shifters with unchanged behaviour: the pass dropped from 145 analyses to 5 (28 s to 6 s locally) for about +2% generic gates.
- **Bounded programs by design.** A PIO state machine cannot take an asynchronous branch while it waits on a data bit, so the I2C slave receiver is fixed to a byte count at assembly time rather than risk going out of step on an early STOP.
- **No PDK cells, no macros.** Memories are Verilog arrays that synthesis maps to flip-flops. See Section 6.3.

4.3 Host tooling

`tools/pioasm.py` assembles stock `pioasm` syntax. `tools/pio_host.py` turns 'load this program, configure the state machine, start it' into a paged flash image, because each PIO register write costs several RV32I instructions and the on-chip execute window holds only 11 instructions. One `tools/build_pio_*.py` builder per demo produces the matching flash image.

5. Protocol coverage

Everything below runs as a PIO program (or as CPU firmware where noted). **All results are simulation.** 'Evidence' names the tests that back each row; Section 7 explains the methods. Word counts are of the 32 shared instruction words.

Protocol	Program (words)	Status	Evidence
UART TX	`uart_tx`	Done	Cycle-exact 8N1 waveform, integer / fractional / averaged dividers; CPU demo `tb_pio_cpu_uart.v`
UART RX	`uart_rx`	Done	TX+RX loopback, framing-error IRQ, baud tolerance, two-SM loopback
SPI master, modes 0-3	`spi_master`, `spi_cpha1`, `spi_cpol1_cpha0`, `spi_cpol1_cpha1`	Done	Data both ways, SCK idle level and edge count, SCK period, MOSI setup/hold, fast SCK with SYNC_BYP; CPU runs all four modes from one image with data relayed (`tb_pio_cpu_spi4.v`)
I2C master	`i2c`	Done	Write, read, repeated-start register read, NAK to IRQ 0, clock stretching; CPU demo `tb_pio_cpu_i2c.v`
I2C slave, both directions	`i2c_slave_rx` (29), `i2c_slave_tx` (32)	Done; write length fixed at assembly time; SCL period >= 32 clocks (use >= 40)	9 tests plus SCL-speed sweep; CPU swaps the two programs between a write and a read of the same device (`tb_pio_cpu_i2c_slave.v`); 29 mutants all caught
I2C multi-master	`i2c_mm` (27)	Done; **no bus-busy detection**	10 arbitration / clock-sync tests; 4 more document the bus-busy hazard (2 of 8 timings destroy the other master's transaction)
JTAG master	`jtag` (24)	Done	CPU resets the TAP, reads IDCODE, writes and reads back the USER register: 95 TCK pulses, pins-only TAP model checks every state
SWD host	`swd` (15)	Done at `pio.v` level; no CPU demo	18 tests against a protocol-strict SW-DP + MEM-AP model; 20 mutants all caught
CAN 2.0A/B	`can_tx` (20) + `can_rx` (10)	Done in simulation; CPU-assisted (CRC-15 and bit stuffing run on the CPU)	25 tests against a CAN bus model with controller nodes (hard sync, 75 % sampling, destuffing, CRC, ACK, arbitration): 8 frame types with TXD edges on the 16-tick grid, arbitration lost / won at every identifier bit, bit errors, loop-delay margin, RX word for word against a Python reference model; CPU-driven bench; 52 mutants all caught
Low-speed USB host	`usb_ls` (29)	Done; receive tolerance -0.5 % to +0.8 %	7 tests against a low-speed device model; CPU queues an IN token and halts, PIO does the transaction (`tb_pio_cpu_usb.v`)
PS/2 receiver	`ps2_rx` (8)	Done, CPU-driven end to end	7 tests incl. 4-frame burst, FIFO overflow, idle-gap resync; CPU sleeps while a 10 kHz keyboard model types
WS2812 / SK6812	`ws2812` (4)	Done; about 5 pixels per CPU-fed frame	5 tests with exact pulse widths (375 / 875 ns), RGBW; CPU demo with RGBW and 280 us-reset variants
WS2812 long strip	`ws2812_repeat` (15)	Done; border LOW stretched to 1.0-1.5 us	300 pixels from one pair of FIFO words; CPU demo paints 60 pixels alone; 18 mutants all caught

Protocol	Program (words)	Status	Evidence
1-Wire master	`onewire` (31)	Done; standard speed only	7 tests against a device model checking every pulse (reset, presence, slots, READ ROM + CRC-8); CPU demo; 23 mutants all caught
VGA 640x480	`vga_frame` (23) + `vga_line` (9)	Done; 8 colours; 57.1 Hz at 24 MHz	Monitor model counts HSYNC, VSYNC and 525 lines over 3 frames; CPU starts two SMs with one write and halts; 23 mutants all caught
PS/2 CPU bit-bang, ST7789 LCD	CPU firmware	Done (PS/2 only against a slowed keyboard)	`tb_ps2_reader.v`, `tb_ps2_ascii.v`, `tb_st7789_driver.v`
10BASE-T, CAN	-	Not attempted	Ethernet needs a frame buffer and at least 40 MHz

Reprogrammability demo. `tools/build_pio_multi.py` builds one flash image whose firmware reprograms the *same* state machine at run time: UART TX, then SPI mode 0, then I2C. Between phases it disables and restarts the machine, overwrites instruction memory and reconfigures pins, shifting and clock divider. `tb_pio_cpu_multi.v` checks all three waveforms against a UART receiver, an SPI slave and an I2C slave, and that the phases ran strictly in sequence. The firmware then executes EBREAK while the PIO finishes the I2C STOP alone.

6. Physical implementation and portability

6.1 Hardening runs

Run	Design	Result
Sky130, Tiny Tapeout CI (previous target)	CPU-only, 6x2 tiles	53.8 % routing utilisation, 12,548 cells (excl. fill/tap), precheck 15/15, gate-level 11/11
IHP, first run, 1 MHz target	CPU-only, 6x2 tiles	17,795 cells, 0.245 mm2, about 62 % utilisation, 0 DRC / LVS / antenna violations, critical path about 206 ns, 5 max-slew violations (slow corner)
IHP, Tiny Tapeout SG13G2 flow (earlier run, labelled CMOS5L at the time)	CPU + PIO, 6x4 tiles	51.45 % routing utilisation, 34,053 cells (excl. fill/tap, as stated in the README), clean lint, precheck 10/10; the `gds` job took about 2 h 28 min (CI run #11)
IHP, SG13G2 flow, 24 MHz period (41.667 ns)	CPU + PIO, 6x4 tiles	Cells 36,601 incl. 3,605 hold buffers (as stated in `docs/info.md`); identical synthesis, placement and routing to the 1 MHz run because nothing needed repair; extracted netlist has 3,433 flip-flops
IHP SG13 CMOS5L flow (current target, run #90)	CPU + PIO, 8x4 tiles, density 58 %	38,504 cells excl. fill/tap, 41.8 % utilisation, wire length 2,478,845 um, clean lint; gds 4 h 34 min; precheck 9/9; gate-level 13/13. At 6x4 detailed routing did not converge; at density 60 % the precheck found one Metal2 spacing violation (M2.b, about 1 nm corner to corner)

The cell counts 34,053 and 36,601 come from two different documents and are quoted as each states them. They have not been reconciled against the raw run summary and may differ in what they count; both are therefore approximately 35,000 cells.

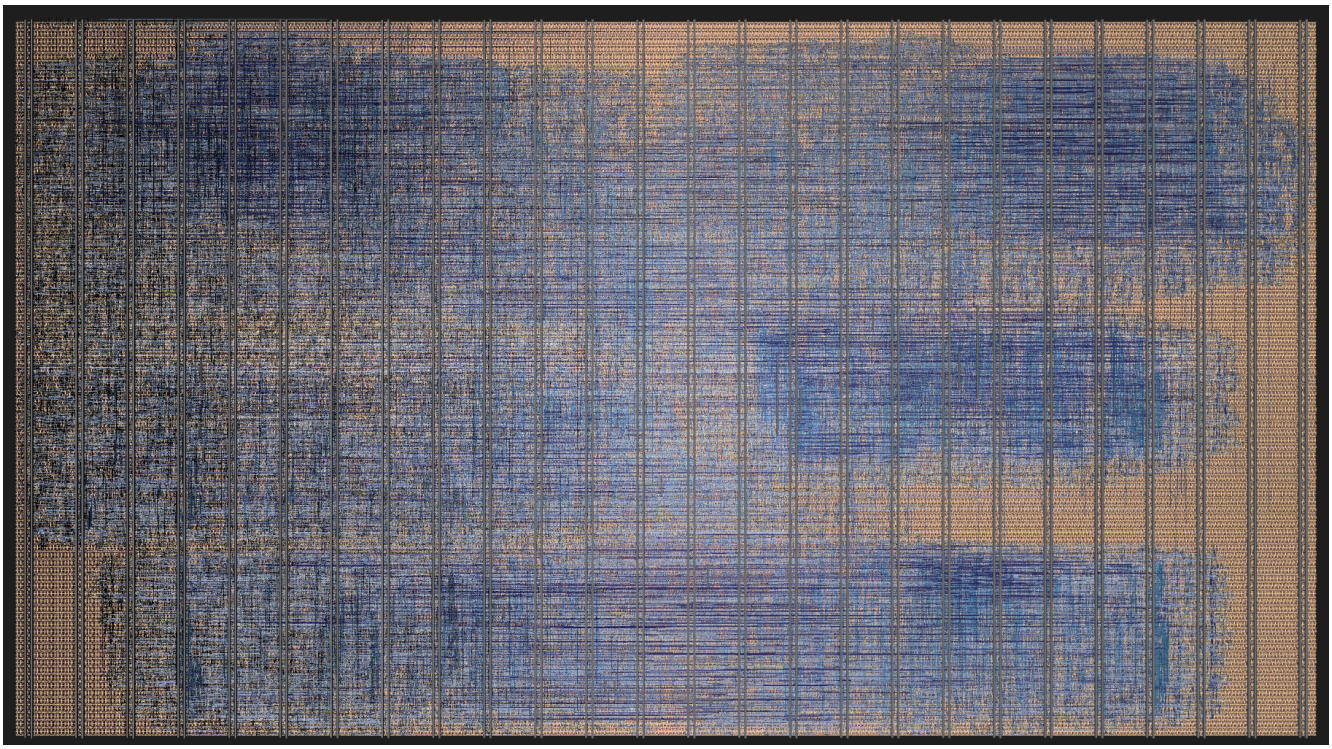


Figure 2. Layout render of the placed-and-routed design from the earlier 6x4 SG13G2 run, as supplied by the author.

6.2 Timing at 24 MHz

Corner	Worst setup slack	Note
Slow (1.08 V, 125 C)	+20.17 ns	Worst path 22.1 ns arrival, register to register; limit about 46.5 MHz, roughly 1.9x margin at 24 MHz
Typical	+27.6 ns	TNS 0 in every corner
Fast	+29.6 ns	Worst hold slack +0.116 ns (this corner); no hold violations

Other checks in that run: 0 Magic DRC, 0 routing DRC, 0 LVS errors, 0 antenna-violating nets. Unchanged design-rule advisories (not failures): 44 max-slew violations (slow corner only), 258 max-fanout and 1 max-cap violation. The 24 MHz clock is chosen so the PIO tick is exactly 12 MHz at CLKDIV 2, which gives low-speed USB its 1.5 Mb/s with 8 ticks per bit.

6.3 Portability: PDK-agnostic RTL, no macros

- **No PDK dependency in the RTL.** `src/*.v` and `*.vh` are plain synthesizable Verilog. No standard cell, I/O cell or other library primitive is instantiated (every instantiated module is defined in `src/`, checked by scanning all module instantiations), and the only conditional compilation is `` `ifndef SYNTHESIS `` around simulation-only code.
- **No macros, no hard IP.** `src/config.json` lists no macros or extra files. All memories (16-bit PIO instruction memory, register file, 48 B RAM) are Verilog arrays that synthesis maps to flip-flops, not SRAM. The cost is area, which is why the instruction memory is 32 words. The gain is that any open flow and PDK can build the same RTL.
- **What carries over and what does not.** The CPU-only version was hardened on Sky130 and moved to IHP without an RTL change. The PIO block was added afterwards and has been hardened on IHP only, so its portability rests on generic RTL, not on a second PDK run. Timing and area figures belong to the IHP run; another PDK needs its own flow run.

7. Verification

7.1 Method and what it can and cannot show

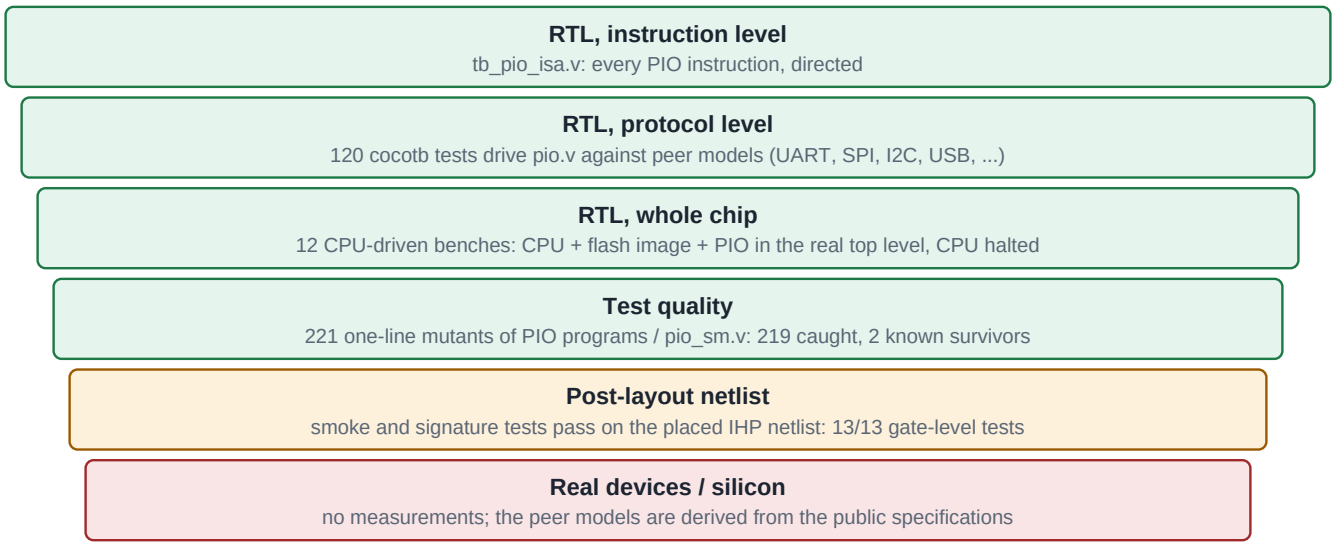


Figure 3. The evidence ladder. Green levels are repeatable in CI; amber is a single post-layout test; red is empty.

The central idea is to test against **peer models that check protocol events, not just waveforms**:

- The I2C slave model flags any SDA change while SCL is high, so START and STOP are checked as events.
- The SPI slave model measures MOSI setup and hold. A model that samples in the same cycle MOSI changes cannot tell a correct program from one whose data and clock edges coincide; swapping the mode programs makes the tests fail.
- The 1-Wire device model checks every pulse width against the specification windows; the SWD target ignores everything until the JTAG-to-SWD switch and checks request parity; the VGA monitor model counts every sync period over three frames.
- **CPU-driven benches** run the real CPU, a flash image and the PIO in the real top level, then check the waveforms from the pins alone, and that the CPU halted before the protocol finished.
- **Mutation sweeps** break a program one line at a time on a temporary copy and require some test to fail. This measures the tests, not the design.

> **Limits of the method.** Every peer model is the author's reading of a specification, not a measured device. A model and a design that share a misreading would pass together. Mutation testing shows that the tests notice changes to the programs; it does not show the programs are right.

What verification found

- Three WAIT encodings in the supplied PIO testbench had the wrong source field (fixed, `CHANGES_feature6.md`).
- The first mutation runs of the 1-Wire and VGA sweeps each exposed a hole in the tests, now closed (`CHANGES_feature15.md`, `CHANGES_feature16.md`).
- The CPU's two-instruction delay loop takes about 274 clocks per pass, not the 14 the helper assumed (the first PS/2 demo slept 4.6 million clocks instead of 170,000).
- Bus-busy hazard in the I2C multi-master program measured: 2 of 8 START timings destroy the other master's transaction, and 'lost arbitration' is also reported in both destructive cases.

- The first real gate-level run failed 3 of 11 tests because a test helper forced a register by hierarchical name, which does not exist in the flattened netlist; the test was fixed, not the design (`CHANGES_feature3_gl_fix.md`). A separate Yosys error, 'multiple edge sensitive events' on one register in `mem.v`, was an RTL coding problem and was fixed in the RTL (`CHANGES_feature4_gl_fix.md`).

7.2 Test inventory

Suite	Count	Runs in	Latest result
On-chip cocotb (`test/test.py` + smoke and signature tests)	13 tests (12 + signature test)	`test.yaml` (as `make`), `gds.yaml` gate-level job	13/13 on RTL (local run) and 13/13 on the placed netlist (gds run #79, 11 Oct 2026, 11 min for the job; run #65: 3 h 11 min)
PIO protocol cocotb (`test_pio_protocols.py`, `test_pio_swd.py`, `test_pio_can.py`)	120 tests (77 + 18 + 25)	`pio-tests.yaml`	120/120, 0 failures, 0 errors, 0 skipped in run #26 (push 79c44fc, results.xml; 25 of them CAN). Run #23: 95/95; 94/94 in run #16, before the I2C short-write test was added
Standalone Icarus benches	32 sources: 15 CPU / peripheral, 13 CPU-driven PIO, 4 `pio.v`-direct	`pio-tests.yaml`	34 runs, all end in PASS (run #26)
Mutation sweeps	221 mutants in 8 sweeps	Manual workflow dispatch; the signature and CAN sweeps were run locally	219 caught, 2 survivors (see 7.3)

7.3 Mutation sweeps

Sweep (`tools/`)	Target	Mutants	Examples of what is broken
i2c_slave_mutation_sweep.py	`i2c_slave_rx.pio`, `i2c_slave_tx.pio`	29	no address compare, ACKs a read request, one byte too many or too few, never drives ACK, any SDA fall counts as START, no clock stretching, ignores the master's NAK
onewire_mutation_sweep.py	`onewire.pio`	23	reset low 464 us (below 480), presence sampled at 37 us, write-0 low 33 us, read sampled 20 us after the fall, data bit inverted, bit count off
vga_mutation_sweep.py	`vga_frame/line.pio`	23	HSYNC 95 ticks, VSYNC one line, 479 picture lines, bar 79 ticks, no black after the last bar, palette not reloaded each line
ws2812_repeat_mutation_sweep.py	`ws2812_repeat.pio`	18	0-bit high 2 ticks, pixel never ends, run never ends, count word ignored, line left HIGH while idle
swd_mutation_sweep.py	`swd.pio`	20	read sampled one tick early, SWCLK high 3 ticks, SWDIO stays an output, read flag inverted, nothing pushed, data two bits per clock
can_mutation_sweep.py	`can_tx.pio`, `can_rx.pio`, `can_frame.py`	52	ACK slot one tick long, first RX sample one tick early, CRC polynomial, and others; all caught in the final run (three survivors in the first run led to new tests)
gl_smoke_mutation_sweep.py	smoke-test programs and `pio_sm.v`	16	no invert, no bit-reverse, no push, wrong IRQ, 3 pulses, wrong widths, jmp x-- not decrementing
gl_sig_mutation_sweep.py	signature-test programs, `pio_sm.v`, `pio.v`	40 (38 caught)	shift direction, autopush threshold, out exec ignored, jmp conditions inverted, wrong IN_BASE, side-set not PINDIRS, side-set enable ignored, no synchroniser bypass, one trip short. Survivors: autopull threshold boundary, STATUS TX/RX select

7.4 Post-layout (gate-level) tests

`test/test_pio_gl.py::test_pio_postlayout_echo_and_pulses` runs in the Tiny Tapeout gate-level job. Through the real boot path (GPIO bootloader, hand-off stub, flash) it runs a 22-page image built by `tools/build_pio_gl_smoke.py`. SM0 (`gl_echo.pio`) pulls a word, inverts and bit-reverses it and pushes it back. SM1 (`gl_pulses.pio`) waits for IRQ 0 and sends bursts of 4 pulses on pad `uo_out[7]`. The CPU feeds 0x55, 0x01, 0xF0; the test checks the three echoes in order, exactly 12 pulses each 4 clocks high with a 10-clock period inside each burst, a low pad at the end, and the expected `uio_oe`.

Result (smoke test): the first gate-level run passed 12/12 tests in 2,587 s (the smoke test is about 10 s on RTL); in run #65 it took 3,444 s. That makes the job about 50 minutes instead of about 6 for the smoke test alone. Run time depends on the hosted machine, so treat the figure as one measurement.

Exercised by the smoke test	NOT exercised by the smoke test
pull block, mov x ~osr, mov isr ::x, push block, irq set 0, wait 1 irq 0, set x, set pins with delay, jmp x--; two state machines (CLKDIV 1 and 2); PIN_OWN on pin 7; CPU-to-PIO bus handshake; both FIFO directions	in; every out form; set pindirs; side-set; every jmp condition except x--; wait on pins; out exec / mov exec; mov from pins or status; autopush / autopull and thresholds; left-shift direction; irq wait / clear / relative; input synchronisers; **pads 8 and 9 (uio_oe never changes)**

> **Consequence.** The I2C, USB and SWD programs rely on exactly the instruction forms and open-drain pads that the smoke test never touches. The signature test below covers those forms and pads and has passed on the placed netlist; the I2C, USB and SWD programs themselves have still not run there.

Signature test (feature 20). `test/test_pio_gl_sig.py::test_pio_postlayout_signature` is a second test in the same two jobs (`COCOTB_TEST_MODULES = test,test_pio_gl,test_pio_gl_sig`). The CPU loads `gl_sig0.pio` (17 words) and `gl_sig1.pio` (14 words) with `tools/build_pio_gl_sig.py` and owns all ten pins; the test compares the pads with golden values worked out by hand and confirmed on RTL. SM0 shows each result on `uo_out[7:0]`: set, mov (plain, ~, ::), in pins / x (left shift), out pins / y (left shift), pull, push, irq set rel, jmp x--, and, through eight host-supplied out-exec words, mov from ~y / isr / status / y, set pins, in y, jmp x!=y, jmp !x (not taken), jmp pin, jmp y--, wait gpio and wait pin with IN_BASE (a taken jump skips a sentinel that would show on the pads). SM1 drives the open-drain pads 8 and 9 with side-set PINDIRS, reads them back through the input synchroniser (pad 8 bypassed, pad 9 synchronised) and uses autopull (threshold 16) and autopush (threshold 8). **Result (signature test):** passed on the placed netlist in gds run #65 (5-6 Oct 2026): gate-level 13/13, 7,564 s for this test; after the change below, gds run #79 (11 Oct 2026) took 164 s for it. The `gl_test` job took 3 h 11 min and the whole workflow 5 h 52 min. After that run the test image was changed to write QSPI_CTRL = 0 first; on an RTL run that mimics the gate-level divider the test needs 120,000 clocks instead of 4.40 million, with the same golden values (Section 7.5). The test checks the exact `uo_out` sequence, the exact (output enable, output value) sequence on the pads with testbench pull-ups, the RX words (0x93, 0xC3, 0x5A and the pushed ISR) and the IRQ flags.

Evidence for the signature test	Result
RTL (local run, 5 Oct 2026)	Passes; full `make` in `test/` 13/13; about 112,000 clocks
Yosys-synthesized netlist of `src/` (generic gates, hierarchy kept; not the hardened IHP netlist)	Passes
Cost on the placed netlist (gds run #65)	7,564 s and 4.40 million simulated clocks, against 3,444 s and 1.97 million for the smoke test in the same run: about 2.2 times (the earlier RTL-based estimate was 1.9 times)
Mutation sweep (40 mutants)	38 caught. Survivors: autopull threshold boundary; STATUS TX/RX select
Run on the placed IHP netlist	**Passed** (gds run #65, 5-6 Oct 2026, push 79f0997): gate-level 13/13, 0 failures; `gl_test` 3 h 11 min, whole workflow 5 h 52 min. Repeated 11 Oct 2026 (run #79, push ed772fa, shortened tests): 13/13, `gl_test` 11 min 1 s, whole workflow 2 h 46 min 38 s. On the SG13 CMOS5L flow at 8x4 (run #90, push 16b8345, 12-13 Oct 2026): 13/13, `gl_test` 8 min 1 s, whole workflow 9 h 22 min

Still not covered by either test: jmp !y, out pindirs / pc / isr, mov exec / pc / isr / osr destinations, push iffull, pull ifempty, a wait that really stalls on a pin, and the latency difference between bypassed and synchronised pins (the delays leave margin).

7.5 Continuous integration

Workflow	What it runs	Last result
`test.yaml`	`make`: on-chip cocotb tests including the PIO smoke and signature tests (RTL)	Result not reviewed for this document
`gds.yaml`	Synthesis, place and route, precheck, gate-level tests	Run #65 (push 79f0997, 5-6 Oct 2026): success, 5 h 52 min; gds 2 h 40 min, gl_test 3 h 11 min, precheck 8 min (the total is about gds plus gl_test, so precheck overlaps); gate-level 13/13. Run #79 (push ed772fa, 11 Oct 2026, after the QSPI change): success, 2 h 46 min 38 s; gds 2 h 35 min, precheck 8 min 6 s, gl_test 11 min 1 s, viewer 20 s; gate-level 13/13. Earlier run: 12/12, 2,587 s of test time. Run #90 (CMOS5L flow, 8x4, push 16b8345, branch try-density58): success, 9 h 22 min 36 s; gds 4 h 34 min, precheck 4 h 47 min (9/9 checks), gl_test 8 min 1 s, viewer 50 s; gate-level 13/13. The CMOS5L precheck is slow because the KLayout DRC takes about 4.8 h on the 8x4 die
`pio-tests.yaml`	`make standalone-tests`: 32 benches + 120 protocol tests; mutation sweeps only on manual dispatch	Run #26 (push 79c44fc): success, 10 min 5 s, 120/120, 34 bench logs PASS. Run #23 (push 22f824b): success, 9 min 15 s, 95/95, 33 bench logs PASS. Run #16 (push 79f0997): success, 6 min 25 s, 94/94, 33 bench logs PASS. Run #10 (03013ea): 4 min 28 s, 94/94. Run #2 (5de56c9): 76/76, 9 min 4 s
`fpga.yaml`, `docs.yaml`	FPGA build, documentation build	Not reviewed for this document

Four runs of the same workflow took 9 min 4 s, 4 min 28 s, 6 min 25 s and 9 min 15 s, so quoted run times depend on the hosted machine. Before `pio-tests.yaml` *existed on GitHub, the PIO benches and the 76-test protocol suite had run only in local simulation, outside GitHub; the first GitHub run (4 Oct 2026) is the first independent confirmation.*

Time budget of the hardening flow. In run #65 the `gds` job took 2 h 40 min and the `gl_test` job, which starts when `gds` ends, took 3 h 11 min (`precheck` and `viewer` run alongside). The author states that the six-hour limit applies to `gds` plus `gl_test` together; the sequence took 5 h 52 min, 8 minutes under 360, which the author treats as too little buffer, and the organisers' FAQ also warns that a design close to full can hit the limit. **Cause of the long gate-level run:** QSPI_CTRL resets to sys_clk/128, and the testbench's force of the divider to 0 does nothing on the flattened netlist, so the smoke test (1.93 million clocks) and the signature test (4.40 million) fetched every flash byte at the slowest speed. **Change (11 Oct 2026):** both flash images now begin with a store of 0 to QSPI_CTRL (0xFB), so the CPU fetches at sys_clk/2 on the netlist too. Measured on RTL with the force disabled (`EMULATE_GL_QSPI=1`): smoke 1.93 million to 40,200 polling clocks, signature 4.40 million to 119,920 clocks, results unchanged. **Result in CI (gds run #79, push ed772fa, 11 Oct 2026):** success, 13/13 gate-level tests, workflow 2 h 46 min 38 s: `gds` 2 h 35 min, `precheck` 8 min 6 s, `gl_test` 11 min 1 s, `viewer` 20 s. The smoke test took 111 s (3,444 s before) and the signature test 164 s (7,564 s before). Gds plus gl_test was about 2 h 46 min on the SG13G2 flow, a buffer of about 3 h 14 min under the six-hour limit. On the CMOS5L flow (run #90) gds plus gl_test is 4 h 34 min plus 8 min, about 4 h 42 min, so the buffer is about 1 h 18 min; an earlier CMOS5L gds run took 5 h 47 min and two layout variants (density 55 %, clock 41 ns) hit the six-hour limit, so the margin is thin. The `pio-tests` workflow (run #30, including the five random start-up seeds) passed in 13 min 55 s. Local hardening remains the fallback.

Random start-up contents (11 Oct 2026). `tools/gen_random_init.py` lists with Yosys every flip-flop that has no asynchronous reset (97 assignments: the register file, the RAM words, the PIO instruction memory, the PIO FIFO memory and a few scratch registers), and `test/tb.v` and the standalone benches load random values into them 1 ns after time 0 when built with `RANDOM_INIT=N` (N = seed). Registers that `rst_n` clears are left alone: filling them at 1 ns only made a short pin glitch before the first clock edge, which bench monitors misread as UART and WS2812 traffic (found in the first attempt, which randomised all 225 flip-flops). A probe confirmed the effect (register x5, RAM word 0 and PIO instruction 0 start non-zero). Results: the 13 cocotb tests pass with seeds 1 to 5 (with the gate-level QSPI timing); the 22 Icarus benches that instantiate `tt_um_agila32` pass with seeds 0 to 3 (247 PASS lines each, no FAIL). Continuous integration runs cocotb seeds 1 to 5 and bench seeds 1 to 3. Limits: the 12 benches that drive only the core, the memory block or the PIO block are not randomised; only flip-flops without reset are randomised; values are random, not adversarial.

8. Claims and the evidence behind them

Strength key: **CI** = reproduced in a GitHub Actions run; **Sim** = simulation run by the author, repeatable with the commands in Section 10; **Model** = checked against the author's own peer model only; **Estimate** = derived or extrapolated; **None** = not done.

Claim	Evidence	Strength
RV32I core runs all base instructions	`tb_alu_test.v` (every opcode the assembler wraps), on-chip cocotb tests	CI (on-chip), Sim
Chip reprograms its protocol engines at run time, no host MCU	`tb_pio_cpu_multi.v`, `tb_pio_cpu_spi4.v`, `tb_pio_cpu_i2c_slave.v`	CI, Model
Protocol keeps running with the CPU halted	CPU-driven benches check the CPU halted before the last pulse / byte (UART, I2C, USB, JTAG, VGA, WS2812)	CI, Model
UART, SPI, I2C baseline met	Section 5 rows, 120 cocotb tests	CI, Model
Low-speed USB host transaction works	`usb_ls.pio` against `UsbLsDevice`; no real device	CI, Model
Timing holds at 24 MHz	GDS run: setup +20.17 ns slow corner, hold +0.116 ns fast corner	GDS run (CI)
Design is DRC / LVS clean	0 Magic DRC, 0 routing DRC, 0 LVS errors, 0 antenna violations	GDS run (CI)
Post-layout netlist behaves like RTL	13/13 gate-level tests, including the PIO smoke and signature tests (no protocol program has run there)	CI, narrow
Tests would catch regressions in PIO programs	221 mutants, 219 caught (2 listed survivors)	Sim (manual workflow, local)
RTL is PDK-agnostic and uses no macros	Instantiation scan, config review, Sky130 to IHP move without RTL change (CPU-only)	Sim, review
Fits 8x4 tiles	41.8 % utilisation on IHP CMOS5L at 8x4 (6x4 did not route)	GDS run #90 (CI)
Works against real peer devices	-	None
Works on silicon	-	None
Formal proofs of the PIO FIFO and state machine	formal/ (4 unbounded proofs pass in GitHub run #3; 28 mutants rejected, local run); Appendix B	CI (proofs), Sim (mutants)
Formal proofs of pio.v, mem.v, the CPU and the QSPI engine	-	None
Constrained-random protocol tests	test/test_pio_random.py: 64 tests per seed; UART, SPI, I2C and PS/2 only (Appendix B)	Sim

9. Limitations and open work

These are collected from the project's own documentation.

9.1 Architecture

- Two state machines (RP2040: eight), 32 shared instruction words, pins 0-9 only. Programs that need more words cannot run together: the two I2C slave programs use 29 + 32 words, so a combined register-file slave (write a pointer, repeated START, read) is not possible; the CPU swaps programs instead.
- The CPU has no interrupts and no traps, executes an instruction in 7 clocks, and feeds about one FIFO word per 3000 clocks (flash paging). Protocols must be slower than that per byte, or fit in the FIFOs.
- The address bus is 8 bits wide; the external window is 16 bytes. Reaching the QSPI Pmod's full capacity needs a wider bus throughout the core.
- The generic SPI peripheral on CS2 needs one board modification (a trace cut on the QSPI Pmod). Until then it is inert.

9.2 Protocol-specific

Protocol	Limit
I2C slave	Write direction is bounded to a byte count fixed at assembly time. SCL period must be at least 32 clocks (40 or more advised). No combined slave.
I2C multi-master	No bus-busy detection before START: 2 of 8 timings destroy another master's transaction, and arbitration loss is reported even then. The host must wait for a free bus, but the check and START are separate steps (3000-clock page switch in between). No stuck-bus recovery. The program has 5 spare words; a PIO-side check was not tried.
Low-speed USB	Receive tolerance only -0.5 % to +0.8 % (spec allows +-1.5 %), so RC-oscillator devices may fail on long packets. TX is FIFO-sized (96 bits), so a SETUP + 8-byte DATA0 transfer is not possible from CPU firmware. Host side only; electrical levels not measured.
PS/2	Host-to-device commands (LED set, reset, inhibit) not done; they need open-drain CLOCK/DATA on pins 8-9. The keyboard model is the author's reading of the timing.
WS2812	CPU-fed frames limited to about 5 pixels; the repeat program stretches border LOW times to 1.0-1.5 us, outside the nominal windows. Not tried on a strip.
1-Wire	Standard speed only; no overdrive or strong pull-up; usable clock window about -7.7 % to +15 %.
VGA	8 colours at about 2/3 brightness; 57.1 Hz at 24 MHz (59.94 Hz needs 25.175 MHz); no frame buffer; no real monitor.
CAN	CPU-assisted: CRC-15 and bit stuffing run on the CPU because PIO has no XOR, and the CPU must keep the 4-deep FIFOs fed and drained. No resynchronisation after SOF (node clocks within about 0.2 % faster / 0.47 % slower over a frame), no bus-idle detection, no automatic ACK, no error counters or error frames, no CAN FD. Not tried on a real bus or transceiver; not run at gate level.
JTAG / SWD	SWD: no CPU-driven demo, no multi-drop, no dormant sequence, not tried on a target (needs an external pull-up on SWDIO). JTAG: model TAP only.
10BASE-T, CAN	Not attempted.

9.3 Verification and evidence

- All results are simulation or RTL proof. Nothing is measured on silicon, on a real peer device, or on a real board.
- Peer models are the author's interpretation of each specification.
- Formal proofs: pio_fifo.v and pio_sm.v only. Random tests: UART, SPI, I2C and PS/2 only (Appendix B).
- Only two small PIO tests have run on the placed netlist (smoke and signature); the protocol programs themselves have not (Section 7.4). The gate-level job took 3 h 11 min and the hardening workflow 5 h 52 min in run #65; the

two tests were shortened afterwards and run #79 took 2 h 46 min on the SG13G2 flow; on CMOS5L at 8x4 (run #90) gds took 4 h 34 min and gl_test 8 min (Section 7.5).

- The design hardens at 8x4 tiles only. At 6x4 detailed routing did not converge; at 8x4 with placement density 60 % the precheck reported one Metal2 spacing violation (M2.b, 0.21 um rule, about 1 nm short at one corner), and density 58 % gave a clean precheck. Whether that violation blocks a submission, and the contest area limit (6x4 in the current brief, 8x4 planned maximum), are questions for the organisers.
- Random start-up contents were tested in RTL simulation (cocotb and Icarus benches), not on the placed netlist (Section 7.5).
- No characterised pad numbers exist for CMOS5L yet, so protocol speeds (for example USB at 1.5 Mb/s) are RTL results and make no statement about pad timing.
- The peer models and the tests were also written with AI assistance (Section 11), so they share any misreading of a specification; no real device has been used as an independent check.
- The external-memory path is tested only against a hand-written flash / PSRAM model.
- Quoted CI and gate-level run times vary with the hosted machine.

9.4 Open work, in suggested order

- Extend formal proofs past pio_fifo.v and pio_sm.v (pio.v first) and random tests past UART, SPI, I2C and PS/2.
- With about 1 h 18 min of buffer on CMOS5L (run #90; 3 h 14 min on the earlier SG13G2 flow), a post-layout I2C transaction test or other gate-level tests can still be added, one at a time; re-check the total after each addition.
- Run the random start-up test on the placed netlist, and randomise the remaining core-only and PIO-only benches.
- An FPGA demonstration against one real peer device (a keyboard, a USB device or an I2C sensor).
- A PIO-side bus-free check for the I2C multi-master program; per-transition resync for USB receive.
- A CPU-driven SWD demo, and a decision on 10BASE-T (the most visible stretch goal, but only if it can be finished and verified).
- Step 3 of the original plan: a PIO glyph expander plus renderer tying the PS/2 keyboard to the ST7789 display.

10. Reproducing the results

Requirements: Icarus Verilog (`apt-get install iverilog`), Python 3.11 and `pip install -r test/requirements.txt` (cocotb 2.1.0 was used).

```
cd test
make                    # on-chip cocotb: self-test, demo counter, bootload-and-run, PIO smoke and signature tests
make standalone-tests   # 32 Icarus benches, then the 120 cocotb protocol tests (Makefile.proto)
make -f Makefile.proto COCOTB_TEST_FILTER=test_onewire_read_rom   # one protocol test

# mutation sweeps (each runs the tests once per mutant; add --list to see the mutants)
python3 tools/i2c_slave_mutation_sweep.py   # 29 mutants
python3 tools/onewire_mutation_sweep.py     # 23
python3 tools/vga_mutation_sweep.py         # 23
python3 tools/ws2812_repeat_mutation_sweep.py   # 18
python3 tools/swd_mutation_sweep.py         # 20
python3 tools/gl_smoke_mutation_sweep.py    # 16
python3 tools/gl_sig_mutation_sweep.py      # 40 (38 caught)
```

The longest single protocol test (`test_onewire_clock_tolerance`) takes about 80 s. In CI, `pio-tests.yaml` runs on every push and fails on any `FAIL` log or `<failure` in `results.xml`. The gate-level run needs the Tiny Tapeout hardening flow and happens in the `gds.yaml` workflow.

Repository layout

Path	Content
`src/`	RTL: `rv32i_core.v`, `mem.v`, `qspi_shared_engine.v`, `pio.v`, `pio_sm.v`, `pio_fifo.v`, `tt_um_agila32.v` (about 2,900 lines with headers), generated `boot_rom_body.vh`, `config.json`
`pio/`	25 PIO programs (.pio files)
`tools/`	Assemblers (`asm_pineapple.py`, `pioasm.py`), host library (`pio_host.py`), flash-image builders (`build_*.py`), protocol helpers, mutation sweeps, `sta.py` (quick pre-layout timing estimate)
`test/`	Cocotb tests, 31 standalone benches, peer models (`pio_tb_lib.py`), flash images, Makefiles
`docs/info.md`	Project datasheet (how it works, how to test, pinout, per-protocol detail)
`CHANGES_feature*.md`	One record per feature, including limits and mutation results
`MIGRATION_TO_IHP.md`	Sky130 to IHP notes
`.github/workflows/`	CI workflows

11. Use of AI tools

The competition rules allow AI tools and ask for a description of how they were used. This section gives that description.

- **Tool.** Claude, an AI assistant from Anthropic (the byline names Claude Sonnet 5.5), used through Claude Code and the Claude app.
- **What the author did.** Set the goals and the feature order, decided what was in scope, ran the GitHub Actions workflows, supplied the CI results, logs and layout images used in this document, and approved every change.
- **What the assistant did.** Helped write and extend the Verilog, the PIO programs, the assemblers and tools, the tests, benches and mutation sweeps, this document, the browser simulator, the FPGA core test and the C flow, and read the CI logs to extract the figures quoted here.
- **How the output was checked.** Generated code was not accepted on its own. It was run against the test suite (120 cocotb tests and 34 bench runs in run #26), the peer models, 221 one-line mutants, and the gate-level tests. Every pass / fail figure in this document comes from a CI log or result file named in the text, not from the assistant's own claim.
- **Errors in the assistant's output that were found.** An FPGA capacity estimate that was too low (corrected after synthesis: the core plus memory needs 118 % of the device, so the whole design does not fit), a QSPI timing estimate that was too low, and archives that omitted changed files. Each was caught by running the tool or by the author's review and then corrected.
- **Limit of the checks.** The peer models and the tests were written with the same assistant, so a misreading of a specification could appear in both. A real peer device is the independent check, and none has been used.
- **Third-party code and names.** The PIO block is an independent implementation of the RP2040 PIO instruction set, written from the public datasheet; it is not affiliated with or endorsed by Raspberry Pi Ltd, and the names Raspberry Pi, RP2040 and Pico are theirs. Five programs in `pio/` are Raspberry Pi Pico SDK or pico-examples programs (BSD 3-Clause) and four more derive from them; `NOTICE.md` lists them with the licence text. The other PIO programs, the CPU, the tools and the tests are original to this project.

Appendix A. Feature history

The project grew by numbered features; each has a `CHANGES_featureN.md` with what changed, the tests, the mutation checks and the stated limits.

#	Feature
1	Halted-status PIN_MUX mode (ported from AgilA8)
2	Boot timeout, automatic flash fallback (ported)
3	Variable SPI clock divider (ported); followed by RTL and gate-level fixes (3/11 to 11/11)
4	General-purpose SPI peripheral on CS2 (ported); followed by a gate-level fix for a multiple-edge-event error
5	Dual-port RAM area merge: investigated, not applicable
6	PIO block: firmware-defined protocol emulation (UART, SPI mode 0, I2C), 26 cocotb tests at the start
7	SPI master modes 2 and 3 (CPOL = 1)
8	CPU-driven demo of all four SPI modes
9	CPU-driven JTAG (IEEE 1149.1)
10	PS/2 receiver on the PIO, and an honest redesign of the keyboard-to-display step
11	Low-speed USB host engine; core clock raised to 24 MHz
12	I2C slave (both directions) and arbitrating multi-master
13	CPU-driven top-level demo of the PS/2 receiver
14	WS2812 / SK6812 LED strips
15	1-Wire master
16	VGA 640x480 colour bars on the Tiny VGA Pmod
17	WS2812 extras: repeat-colour program, RGBW and 280 us reset variants; PIO test workflow
18	Post-layout (gate-level) PIO smoke test
19	SWD (ARM Serial Wire Debug) host on the PIO
20	Post-layout (gate-level) PIO signature test
21	CAN 2.0A/B on the PIO (transmitter with bus readback, raw bit capture), CPU-assisted

Sources used for this document

- `README.md`, `docs/info.md`, `info.yaml`, `src/config.json`, `MIGRATION_TO_IHP.md` and `CHANGES_feature*.md` of the 5 Oct 2026 repository snapshot.
- GitHub Actions results: `pio-tests` run #16 (push 79f0997), run #10 (push 03013ea) and the earlier run #2 (push 5de56c9), the `results.xml` and bench logs of those runs, and the Tiny Tapeout gate-level `results.xml` (first run: 12 test cases; gds run #65 of 5-6 Oct 2026: 13 test cases with per-test times, 0 failures).
- Competition brief: Jane Street, 'Protocol emulator ASIC competition' (blog.janestreet.com/protocol-emulator-asic-competition/), and the organisers' update and FAQ email of October 2026 (forwarded by the author on 10 Oct 2026).
- Added 30 Oct 2026 (Appendix B): `CHANGES_oct30_formal.md` , `CHANGES_oct30_random_tests.md` , `formal/README.md` and the GitHub Actions `formal` run #3 (push `f03db8d`) with its `formal-logs` artifact.

Appendix B. Formal proofs and random tests

Added 30 Oct 2026, after the 7 Oct 2026 snapshot this document describes. Nothing in `src/`, `pio/`, `info.yaml` or `src/config.json` changed, so the hardened design and gds run #90 are unaffected. The cover and Sections 1, 2, 8 and 9 point here.

Summary. Four unbounded formal proofs of the PIO FIFO and of one PIO state machine pass in GitHub run #3 (30 Oct 2026, 2 min 20 s). They cover two modules, `pio_fifo.v` and `pio_sm.v`, as RTL. They do not cover `pio.v`, `mem.v`, the CPU core or the QSPI engine, and they say nothing about silicon. The protocol programs are still verified by the peer models, benches and mutation sweeps of Section 7.

B.1 Formal proofs of the PIO FIFO and state machine

The proofs use SymbiYosys with the `abc pdr` engine. They are **unbounded**: they hold for every input sequence with no depth limit, not only for the cycles a simulation visits. The design files in `src/` are **not modified**: `formal/instrument.py` writes a copy in `formal/build/` with the property file inserted before `endmodule`, so the properties can see internal registers. The z3 and boolector SMT engines were too slow for these designs.

Module and task	Property
`pio_fifo.v` (TX and RX queue of every state machine), task `props`	**P1** level never above the depth; full / empty flags agree with the level; never both. **P2** level next = level + push - pop; push into a full FIFO and pop of an empty FIFO change nothing; clear and reset empty the queue. **P3** read / write pointers agree with the level. **P4** data integrity: a word pushed is the word popped, in order, bit-exact, never lost or duplicated.
`pio_fifo.v`, task `equiv`	The circular buffer behaves exactly like a shift-register reference queue for every push / pop / clear / reset sequence.
`pio_sm.v` (one state machine; any instruction word, any configuration, any FIFO / pin / IRQ state, reset at any time), task `core`	**S1** never pops an empty TX FIFO, never pushes into a full RX FIFO (it would silently drop the data). **S2** OSR / ISR bit counters never exceed 32. **S3** quiet when not ticking (disabled, between divided-clock ticks, in a delay slot): no pin or direction driven, no pop or push, no IRQ flag raised or cleared. **S4** pin and direction values only where the mask says the pin is written. **S5** a stalled instruction keeps the PC; a host-forced instruction that does not jump does not move it; no tick, no movement. **S6** delay slots last exactly the programmed number of ticks. **S7** restart clears the OSR / ISR counters, ISR, delay, phase and EXEC state.
`pio_sm.v`, task `clkdiv`	**S8** the clock divider ticks exactly every N clocks for an integer divider N; the accumulator stays below the divisor. Proved for a fixed divider below 8.0 only.

Table B1. Properties proved (sources: `fifo_props.vh`, `pio_fifo_formal.v`, `sm_props.vh`, `formal/README.md`).

B.2 Result on GitHub

Workflow `formal.yaml` (manual, `workflow_dispatch`; optional input `mutants`), **run #3**, started by hand on 30 Oct 2026, push `f03db8d` on `main`. Status: success. The job took 2 min 16 s and the whole run 2 min 20 s; the four proofs need 35 s of solver time, the rest is set-up. Artifact `formal-logs` (8.33 KB): the four SymbiYosys logs, each ending `DONE (PASS, rc=0)`. One annotation, GitHub's Node.js 20 deprecation warning, does not affect the result. This run did **not** include the mutant check (B.3).

Proof (SymbiYosys task)	Engine	Result	Elapsed clock time
`pio_fifo props`	abc pdr	PASS	4 s
`pio_fifo equiv`	abc pdr	PASS	10 s
`pio_sm core`	abc pdr	PASS	1 s
`pio_sm clkdiv`	abc pdr	PASS	20 s

Table B2. Result of each proof in GitHub run #3 (from the four logs in the `formal-logs` artifact).

B.3 Do the proofs bite?

`formal/check_mutants.py` breaks a copy of the design 28 ways, one at a time: FIFO pointer and flag errors, popping an empty FIFO, pushing into a full one, a counter that is not capped, a delay slot that executes, a stalled instruction that moves the PC, a divider that ticks one clock late, and others. Every harness accepts the real design and rejects every broken copy. Two first-draft mutants that were not rejected were equivalent changes and were replaced by changes that alter behaviour. This check was run **locally** (`bash run.sh --mutants`, about 4 minutes), not in the GitHub run of B.2. It shows that the properties notice broken designs, not that the set is complete.

B.4 Limits of the formal work

- Only `pio_fifo.v` and `pio_sm.v` are covered. **Not covered formally:** `pio.v` (register interface, pin multiplexing, IRQ flags), `mem.v`, the CPU core and the QSPI engine. They rest on the cocotb and Icarus tests, the random tests and the mutation sweeps of Section 7.
- S8 (clock-divider period) is proved for a fixed divider below 8.0 (`CLKDIV_SMALL`); the solver does not reach the full 24-bit range. S3 to S7 have no such limit.
- The properties say what must never happen and some exact timing rules. They do not prove that a given PIO program implements a protocol correctly; the protocol tests do that. As with the peer models, the properties are the author's reading of the intended behaviour.
- The asynchronous reset is modelled as SymbiYosys does (`async2sync`): it acts in the cycle it is low; the proofs start in reset and let it fall in any later cycle. FIFO memory has no reset on silicon, so the proofs start with arbitrary contents.
- Toolchain: `pio_sm.v` indexes `side_fld[val_bits]` past bit 4 when `side_count` is 5 without the enable bit (the result is masked). Recent Yosys turns that into a constant `x` and `write_aiger` stops, so `pio_sm.sby` lowers the shifts and uses `$anyseq` after `prep`. `src/pio_sm.v` is unchanged.
- The proofs are of the RTL. They are not of the placed netlist, and nothing has been measured on silicon.

B.5 Constrained-random protocol tests

`test/test_pio_random.py` runs 64 constrained-random cocotb tests per seed on `pio.v` alone: UART TX; UART RX including break frames and ±1.5 % baud error; SPI modes 0-3 including CLKDIV 1 with the MISO synchroniser bypassed; I2C write, read, register read and wrong address with random clock stretching; PS/2 RX. Each test is checked against a reference model with bit-timing checks. Run: `make -f Makefile.random` (optional `PIO_RANDOM_SEED=n` and `PIO_RANDOM_N=n`).

Seeds 1-6 (384 iterations) passed locally, about 40 s per seed on one core. Against deliberately broken copies of the design (not committed: UART, SPI, I2C and PS/2 timing, OSR shift direction in `pio_sm.v`) every mutant made at least one iteration fail. A new step in `pio-tests.yaml` runs seeds 1-5 on every push; its GitHub result is not reported here. CAN, 1-Wire, JTAG, SWD, USB and WS2812 use directed tests and mutation sweeps only.

B.6 Reproducing

```
cd formal && bash run.sh              # four proofs, about 1 minute
cd formal && bash run.sh --mutants    # also the 28 mutant checks, about 4 more minutes
cd test && make -f Makefile.random PIO_RANDOM_SEED=1   # 64 random tests, one seed
```

Needs Yosys, SymbiYosys and `yosys-abc` (Yosys 0.33 locally; the OSS CAD Suite in GitHub Actions). On GitHub: Actions, `formal`, Run workflow (tick `mutants` for the mutant check).